# Autonomous Obstacle-Avoidance Robot Car

Component Analysis and Working Principles

Lianet Martin • Electrical Engineering

Embedded Systems / Robotics Project

## 1 Project Overview

This document analyzes the components of an autonomous, four-wheeled robot car that detects and avoids obstacles without human control. The vehicle uses a **skid-steer** (differential) drive: the two motors on each side are driven together, and the car turns by spinning one side faster than the other, like a tank.

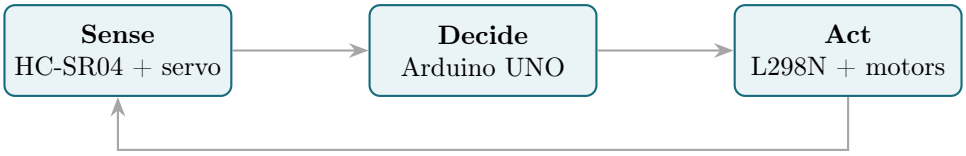
The system is built around a single, continuous control loop: **sense** → **decide** → **act**. An ultrasonic sensor perceives the environment, an Arduino microcontroller interprets that information and decides how to respond, and DC motors execute the resulting movement. A battery powers the entire system. Every component in the sections that follow plays a specific role inside this loop.

| Component | Role | Function in the car |
|---|---|---|
| Arduino UNO R3 | Decide | Central controller; runs the code and unites the system |
| HC-SR04 | Sense | Ultrasonic sensor that measures distance to obstacles |
| Micro servo SG90 | Sense | Rotates the sensor to scan left, center, and right |
| L298N driver | Act | Power stage that drives the motors and sets direction/speed |
| DC gearmotors (×4) | Act | Convert electrical energy into the car's motion |
| Wheels (×4) | Act | Turn rotation into forward movement through traction |
| Battery pack | Power | Supplies energy to both the motors and the electronics |

## 2 System Architecture: The Sense–Decide–Act Loop

The whole vehicle can be understood as a body. Sensors are its senses, the microcontroller is its brain, and the motors are its muscles. A useful lens for this is the idea of a **transducer**: a device that converts one form of energy into another.

- **Input transducers (sensors)** convert a physical phenomenon into an electrical signal the brain can read — e.g. the HC-SR04 turns sound/distance into an electrical signal.
- **Output transducers (actuators)** convert electrical signals from the brain back into physical action — e.g. the motors turn electricity into motion.
- The **Arduino** is not a transducer. It lives purely in the electrical domain, processing incoming signals and deciding on the outgoing ones.




*The loop repeats many times per second.*

This structure — a brain surrounded by transducers that connect it to reality — is the skeleton of essentially every embedded system.

## 3 Decision — Arduino UNO R3

The Arduino is the **hub** of the project: the central node where everything converges. It unites the system on three levels at once.

- **Physically:** it is a star topology. Every wire begins or ends at the Arduino — the sensor's Trig/Echo, the servo's signal, and the driver's control lines all connect to its pins.
- **Electrically:** it distributes 5 V to the components that need it and, crucially, provides the **common ground**: the shared 0 V reference against which every signal is measured.
- **Logically:** its firmware is the conductor of the orchestra. The program stored in memory reads the sensor, decides, and commands the actuators — the sense–decide–act loop lives inside the Arduino.

### 3.1 Inside the microcontroller (ATmega328P)

The Arduino board is a convenient shell around one chip, the ATmega328P, which is effectively a complete tiny computer:

- **CPU** — executes the program instructions, paced by a 16 MHz clock (the resonator).
- **Flash (32 KB)** — non-volatile memory where the program lives; survives power-off, like a disk.
- **SRAM (2 KB)** — volatile working memory for variables while the program runs, like RAM.
- **EEPROM (1 KB)** — small non-volatile memory for data kept between power cycles.
- **ADC** — the analog-to-digital converter that makes the analog pins work.
- **Timers** — generate PWM and drive timing functions such as `delay()` and `millis()`.
- **Ports (B, C, D)** — where the physical pins come from.

### 3.2 Digital vs. analog pins

The UNO exposes **14 digital pins** (0–13) and **6 analog inputs** (A0–A5). The count of 14 follows from the chip's port structure: two of Port B's pins are used by the 16 MHz crystal, leaving 8 (Port D) + 6 (Port B) = 14 digital lines.

- **Digital pins** understand only two states: HIGH (5 V) or LOW (0 V). They can be inputs or outputs. Six of them (marked with a ~) support **PWM**, which is required to control motor speed and servo angle.
- **Analog pins** read a continuous voltage between 0 and 5 V; the ADC converts it to a number from 0 to 1023 (10-bit resolution). They are input-only.

### 3.3 Why a common ground matters

Voltage is never absolute — it is always a *difference* measured against a reference, much like altitude is measured against sea level. Ground (GND) is that agreed 0 V reference. When the Arduino sends a 5 V HIGH on Trig, it creates 5 V *relative to its ground*; the sensor reads that pin relative to *its* ground. Only if both share the same ground do the levels agree and communication works. A shared ground also provides the **return path** that lets current complete its loop. This is why, even when the voltage supplies for the motors and the electronics are separated, all grounds must be tied together.

## 4 Sensing

### 4.1 HC-SR04 Ultrasonic Sensor

The HC-SR04 measures distance the way a bat or a sonar does — by timing sound.

- The Arduino sends a short (10 $\mu$s) HIGH pulse on **Trig** — the command "fire now."
- The sensor emits a burst of 40 kHz ultrasound; the sound reflects off an obstacle and returns.
- The sensor holds the **Echo** pin HIGH for a time exactly equal to the round trip of the sound.
- The Arduino measures that pulse width with `pulseIn(echo, HIGH)` and computes the distance:

$$\text{distance} = \frac{\text{echo time} \times 343\,\text{m/s}}{2}$$

  The division by two accounts for the sound traveling out *and* back.

**Subcomponents.** Two cylindrical **piezoelectric transducers** — one transmitter (T) that converts electricity into sound, one receiver (R) that converts the returning sound into electricity — plus an onboard control circuit that generates the burst and times the echo, and four pins: VCC, Trig, Echo, GND. The *piezoelectric effect* is the physical basis: certain crystals produce a voltage when squeezed, and vibrate when a voltage is applied.

**Role in the car.** The HC-SR04 is the car's "eyes." Typical range is about 2–400 cm. It is mounted on the servo so it can look in different directions rather than only straight ahead.

### 4.2 Micro Servo SG90

A servo is not just a motor — it is a small **closed-loop control system**: a motor that knows its own angle and corrects itself to reach a commanded position (0° to 180°).

**How it works.** The angle is commanded with PWM: a pulse is sent every 20 ms, and the pulse *width* encodes the target angle (~1 ms → 0°, ~1.5 ms → 90°, ~2 ms → 180°). Internally, a potentiometer measures the actual angle, and a control circuit compares it against the commanded angle, driving the motor until the two match, then holding position. This is negative feedback — the foundation of control systems.

**Subcomponents.** A small DC motor, a gear train (trading speed for torque), a potentiometer (position feedback), a control IC, and the output shaft/horn. Three wires: signal, VCC, GND.

**Role in the car.** The servo carries the HC-SR04 and rotates it to scan left, center, and right. It is mounted so that 90° points forward, giving a symmetric sweep to both sides. When an obstacle is detected, the car looks each way, compares the distances, and turns toward the more open direction.

## 5 Actuation

### 5.1 L298N Motor Driver

The Arduino's pins are weak: they output a small control signal, not power, and only in one fixed direction. Motors, by contrast, need large current and must reverse polarity to run in reverse. The L298N sits between them as a **power translator**: it takes the weak signal from the brain and uses it to steer the strong current from the battery to the motors.

**The H-bridge.** Internally, four electronic switches surround the motor in an "H" shape. Closing one diagonal sends current through the motor one way (forward); closing the other diagonal reverses it (reverse); closing none stops the motor. These switches are **transistors**, opened and closed by the Arduino's small signals — which is exactly how a weak signal ends up controlling a strong current.

**Subcomponents.** The L298N chip (which contains *two* complete H-bridges, so it drives two motors independently), an onboard 5 V regulator, screw terminals, a heatsink, and internal protection diodes.

**Pins, in two groups.** Power: battery input and the four motor outputs (OUT1–OUT4). Control (from the Arduino): IN1–IN4 set each motor's direction (which diagonal closes), and ENA/ENB set the speed via PWM.

**Role in the car.** With two channels, the two left motors connect to one channel and the two right motors to the other, producing the skid-steer: both sides forward is straight, one side faster is a turn, opposite sides is a spin in place.

### 5.2 DC Gearmotors and Wheels

At its core, this subsystem *is* the DC motor; the gearbox and wheel are accessories that shape its raw output into useful motion.

**The motor.** By electromagnetism, current through a coil turns it into an electromagnet; fixed magnets push against it and make it spin. A commutator reverses the current in the coil every half-turn so the push always acts in the same rotational direction, producing continuous rotation. The raw output is *fast but weak*.

**The gearbox (reduction).** A train of gears trades speed for torque, converting the fast, weak spin into a slow, strong one — enough to move the weight of the car.

**The wheel.** The rubber tire grips the ground (traction) and converts the shaft's rotation into forward movement. Without grip the wheel would spin in place.

*battery → L298N → motor (fast, weak) → gearbox (slow, strong) → shaft → wheel → the car moves*

In a skid-steer layout the wheels are fixed (no steering wheels); the car turns by the speed difference between sides, so the wheels skid sideways during a turn.

## 6 Power System

A battery stores chemical energy and delivers it as electrical energy, providing both **voltage** (the "push") and **current** (the "flow"). Its key specifications are voltage and capacity (mAh, how much energy it stores). It feeds both the motors (which draw large current, through the L298N) and the control electronics (which need a clean, stable 5 V).

### 6.1 Why a shared supply causes erratic behavior

- Motors draw current in sudden **spikes** — at startup, on direction changes, and under load.
- Every wire and battery has internal resistance. By Ohm's law ($V = IR$), a large current spike produces a voltage drop, so the shared supply momentarily **sags**.

- The Arduino needs a stable voltage (at least ∼4.5 V). If the supply dips below that threshold, the microcontroller **browns out and resets**, restarting the program mid-run — the source of the erratic behavior.
- Motors also inject electrical **noise** onto shared power lines, which can corrupt sensor readings.

### 6.2 The solution

- **Separate the power paths** — give the motors and the logic their own supply lines so they do not compete on the same rail.
- **Always share a common ground** — so all modules reference the same 0 V and current can complete its loops.
- **Add decoupling capacitors** — small reservoirs that absorb the spikes and fill the dips, smoothing the voltage.

## 7 Key Electronics Concepts

These cross-cutting ideas recur throughout the design:

- **Transducer** — converts one energy form to another; sensors are input transducers, actuators are output transducers.
- **Transistor** — an electronic switch with no moving parts, where a small signal controls a large current. It is the building block of the H-bridge and of all digital electronics (the microcontroller itself contains thousands).
- **PWM (Pulse-Width Modulation)** — approximates an analog output by switching a digital pin on and off quickly; used to set motor speed and servo angle.
- **Digital vs. analog** — digital signals carry two levels (HIGH/LOW); analog signals carry a continuous range, read through the ADC.
- **Common ground** — the shared 0 V reference and current return path that lets separate modules communicate.
- **Ohm's law ($V = IR$)** — explains the voltage sag that causes a shared-supply brownout.